

3/7 Heap & Heapsort →Heap →

- Binary tree following two conditions:

- Shape requirement → Essentially complete / simply complete (all its levels are full except possibly the last level)
- Parental dominance → Key at each node is greater than or equal to key at its children

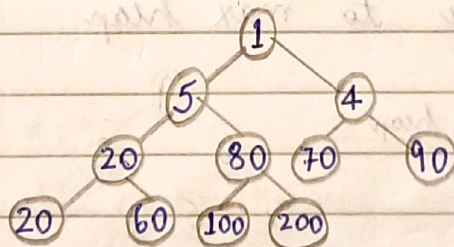
Note Heap with n nodes has height $\log_2 n$

Two types of heaps in DS →

- Min-heap →

- Key present at root node must be less than or equal to keys present at all of its children

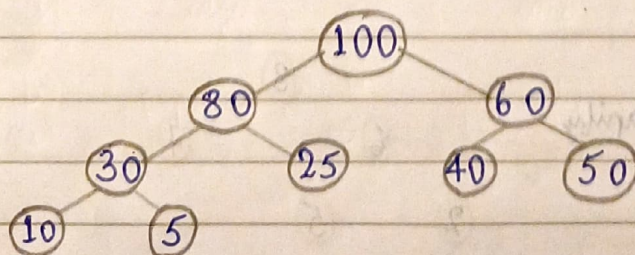
eg



- Max-heap →

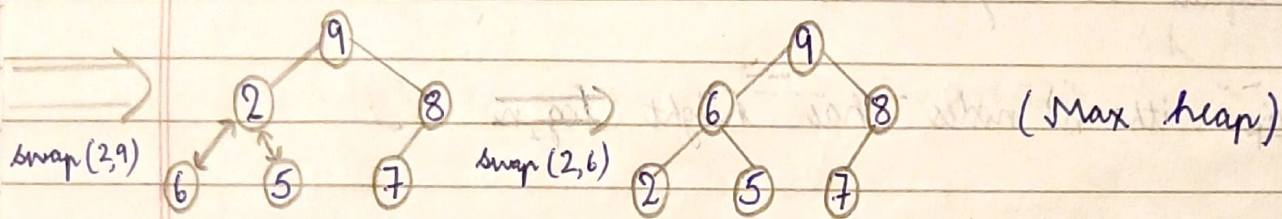
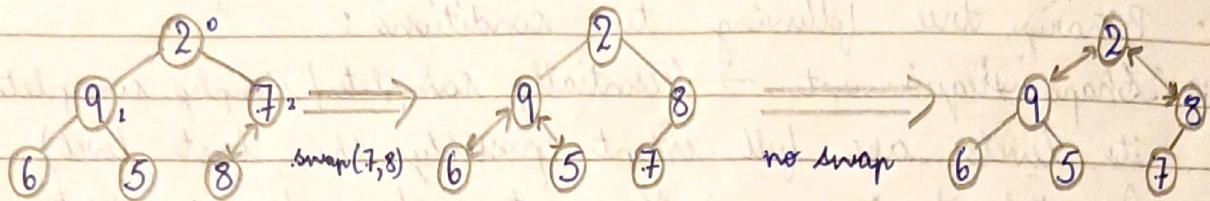
- Key present at root node must be greater than or equal to keys present at all of its children

eg



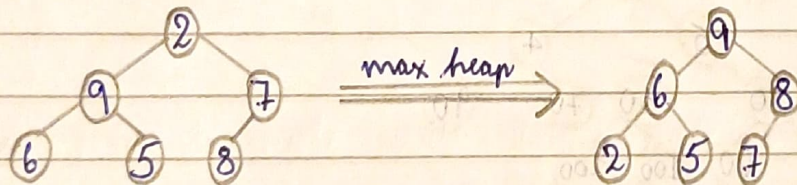
Bottom up construction of heap (max heap) $\rightarrow$

eg $2, 9, 7, 6, 5, 8 \Rightarrow n = 6 \Rightarrow \lfloor n/2 \rfloor - 1 \Rightarrow 3 - 1 \stackrel{i=}{=} 2$ (then keep $i--$)



eg 1 heap sort for $2, 9, 7, 6, 5, 8$

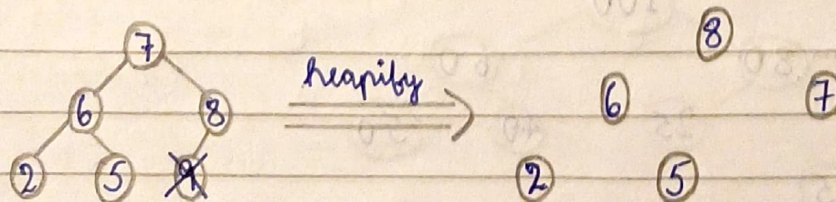
Step I Convert binary tree to max heap



0	1	2	3	4	5
2	9	7	6	5	8

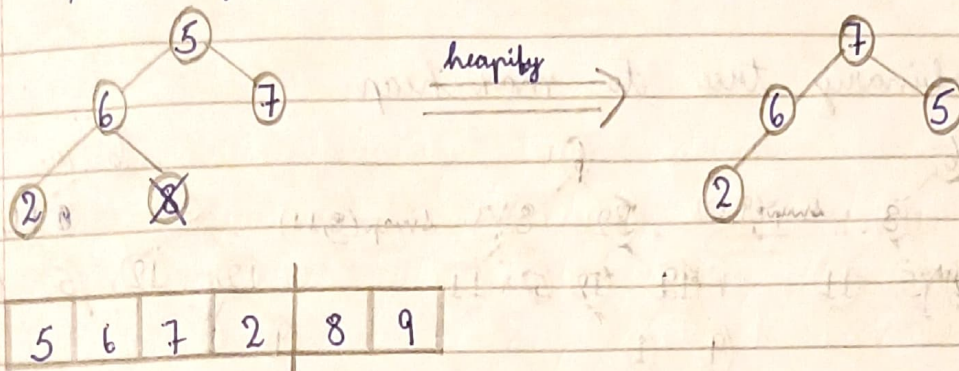
0	1	2	3	4	5
9	6	8	2	5	7

Step II Swap root node with last node & then delete last node

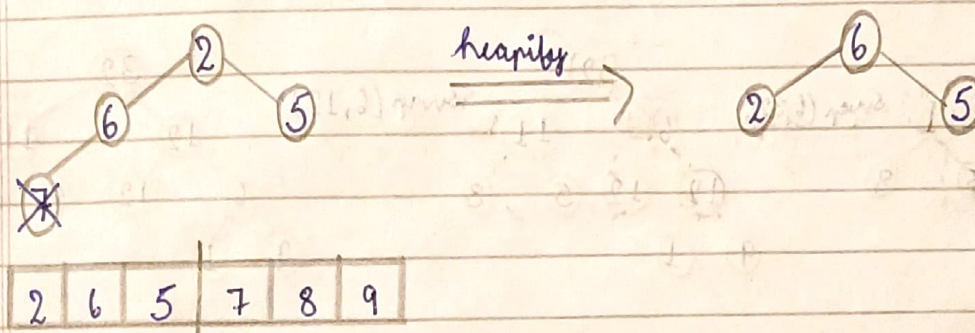


0	1	2	3	4	5
7	6	8	2	5	9

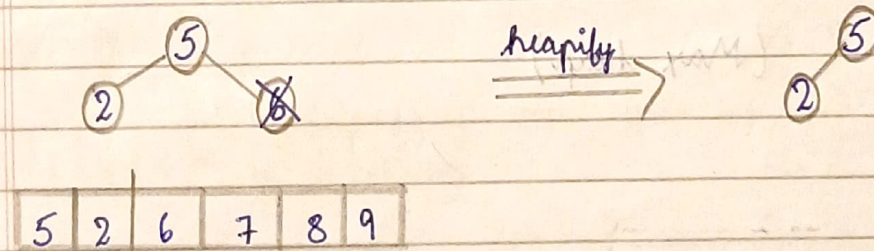
Step III Repeat again & again



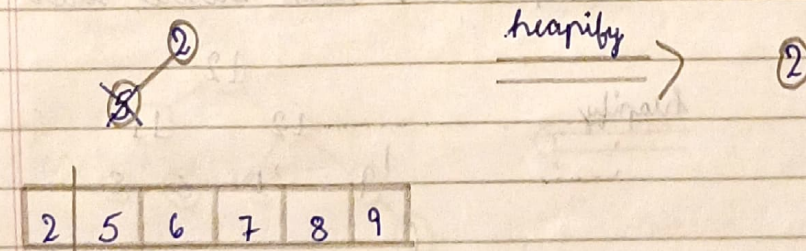
Step IV



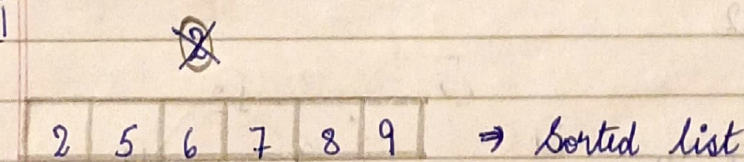
Step V



Step VI

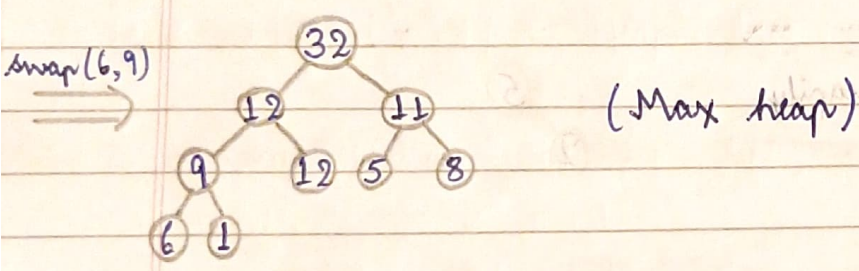
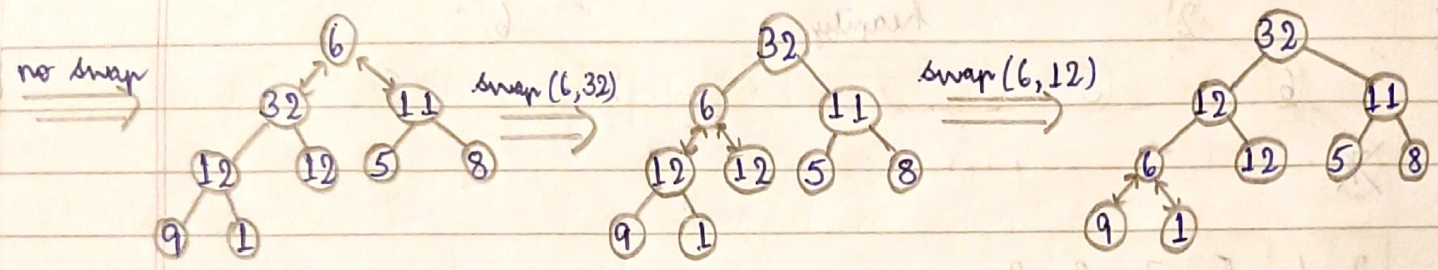
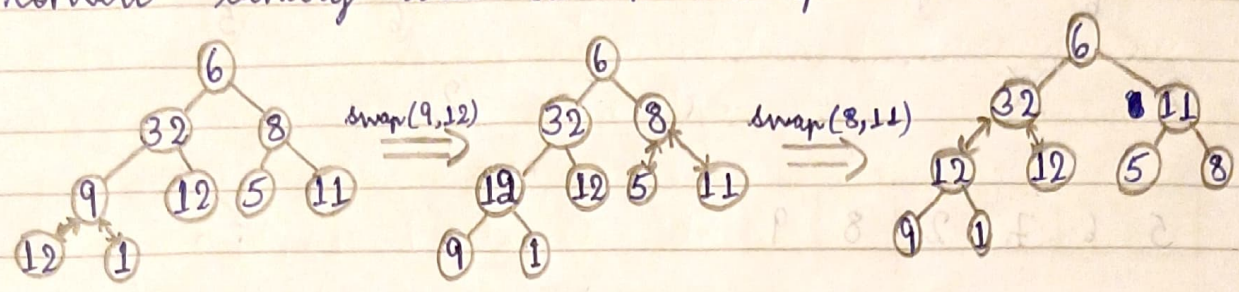


Step VII

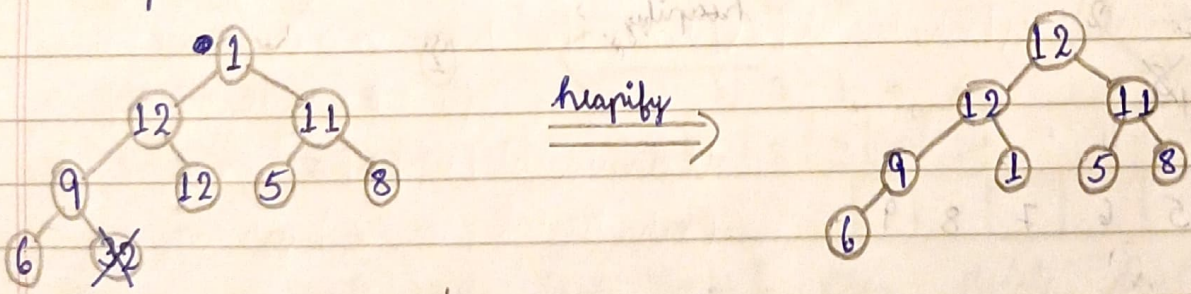


eg Heapsort for 6, 32, 8, 9, 12, 5, 11, 12, 1 $\Rightarrow n=9 \Rightarrow \lceil n/2 \rceil - 1 = 3$

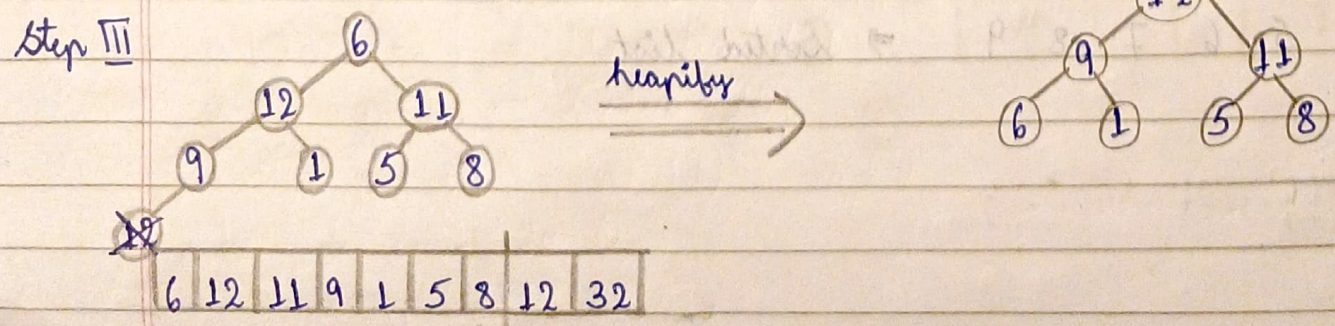
Step I Convert binary tree to max heap



Step II Swap root node with last node & then delete last node

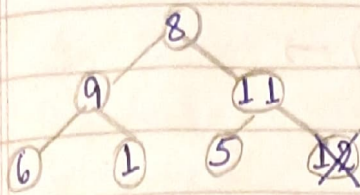


1	12	11	9	12	5	8	6	32
---	----	----	---	----	---	---	---	----

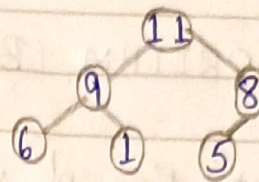


6	12	11	9	1	5	8	12	32
---	----	----	---	---	---	---	----	----

Step IV

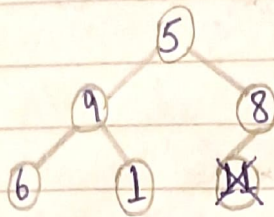


heapify →

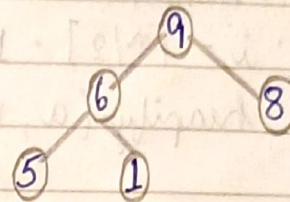


8	9	11	6	1	5	12	12	32
---	---	----	---	---	---	----	----	----

Step V

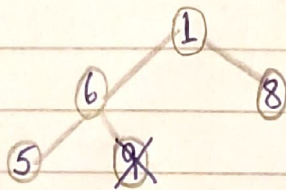


heapify →

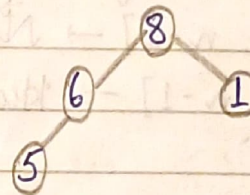


5	9	8	6	1	11	12	12	32
---	---	---	---	---	----	----	----	----

Step VI

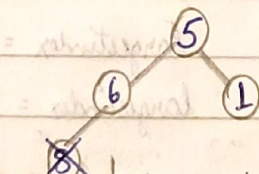


heapify →

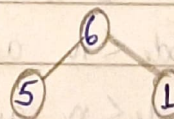


1	6	8	5	9	11	12	12	32
---	---	---	---	---	----	----	----	----

Step VII

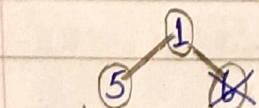


heapify →

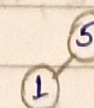


5	6	1	8	9	11	12	12	32
---	---	---	---	---	----	----	----	----

Step VIII

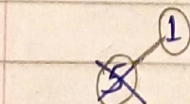


heapify →



1	5	6	8	9	11	12	12	32
---	---	---	---	---	----	----	----	----

Step IX



heapify →



1	5	6	8	9	11	12	12	32
---	---	---	---	---	----	----	----	----

Step X



1	5	6	8	9	11	12	12	32
---	---	---	---	---	----	----	----	----

ALGORITHM BottomUp ($a[0 \dots n-1]$) $\rightarrow$

Purpose Construction of max heap tree

Input $a[0 \dots n-1] \rightarrow$ Array representation of binary tree

Output $a[0 \dots n-1] \rightarrow$ Max heap tree

Algo.

for $i = [n/2] - 1$ to 0 do
 heapify (a, n, i)

ALGORITHM heapify ($a[], n, i$) $\rightarrow$

Purpose Construction of heap tree from given root index

Input $a[0 \dots n-1] \rightarrow$ Non-heap tree

Output $a[0 \dots n-1] \rightarrow$ heap tree

Algo.

largestindex = i

leftnode = $2 * i + 1$

rightnode = $2 * i + 2$

if (leftnode $< n$ and $a[\text{leftnode}] > a[\text{largestindex}]$)

largestindex = leftnode

if (rightnode $< n$ and $a[\text{rightnode}] > a[\text{largestindex}]$)

largestindex = rightnode

if (largestindex $\neq i$)

 swap ($a[i], a[\text{largestindex}]$)

 heapify ($a, n, \text{largestindex}$)

ALGORITHM heapsort ($a[0 \dots n-1]$) $\rightarrow$

Purpose Sort given array using heapsort method

Input $a[0 \dots n-1] \rightarrow$ Unsorted list

Output $a[0 \dots n-1] \rightarrow$ Sorted list

Algo. BottomUp ($a[0 \dots n-1]$)

for $i = n-1$ to 0

 swap ($a[0], a[i]$)

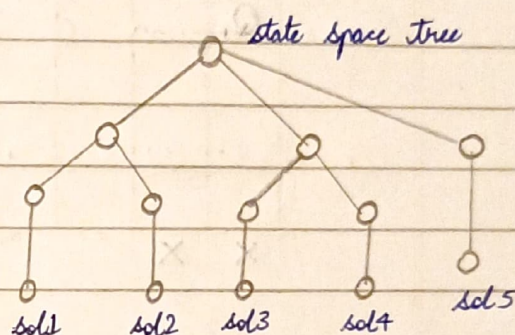
 delete ($a[i]$)

 heapify ($a, i, 0$)

$$\therefore T(n) \in O(n \log n)$$

12/7 Backtracking →

- Used to find all possible solutions ~~to~~ available to problem
- Traverses state space tree by DFS method
- Once it realizes that it has made a bad choice, it undoes last choice by backing up
- Uses recursive based solution
- Has brute force nature

Few Algorithms →

- N - queen problem
- Hamiltonian circuit
- Graph coloring
- Sum of subset

N - Queen problem →

- Consider $N \times N$ chessboard on which we have to place N queens in such a manner that no two queens attack each other
- No two queens can be in same row, column or diagonal

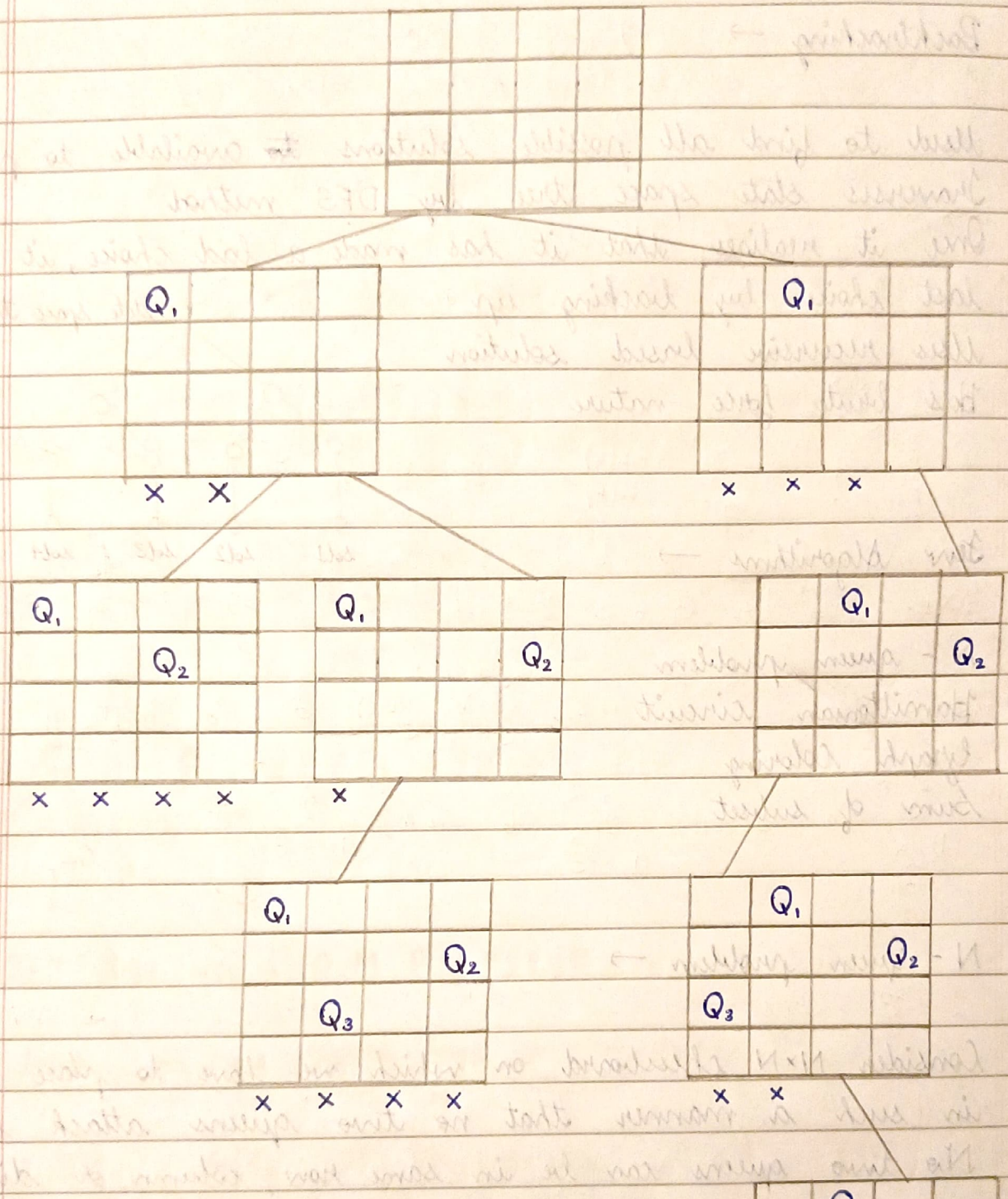
eg $N = 4$

	Q ₁		
			Q ₂
Q ₃			
		Q ₄	

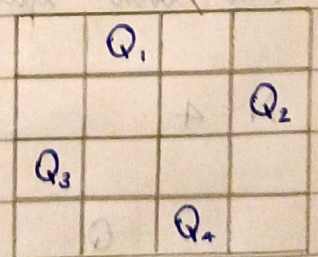
Solution I

		Q ₁	
Q ₂			
			Q ₃
	Q ₄		

Solution II



Solution I



Solution II

